MABEL Data Studio: a Rerun-style visualizer and non-destructive timeline editor for whole-body imitation-learning episodes

MABEL

June 30, 2026

Abstract

The MABEL collection pipeline logs whole-body teleoperation episodes as time-major HDF5 (seven cameras, planar lidar, 59 actuators, full generalized state, body poses, and operator intent at 15 Hz). Data Studio turns that on-disk format into an interactive web tool that (i) *visualizes* an episode like `rviz/rerun` — a 3D rig spawned inside the lidar map it reconstructs, synchronized camera tiles, a dense per-actuator "equalizer", and a checkbox topic tree — and (ii) *edits* episodes like a non-linear video editor — trim, blade, ripple-delete, merge, re-order, and append across multiple source episodes, saved as a new episode. Editing is non-destructive: a timeline is an Edit Decision List (EDL) of source segments, and source `.h5` files are never mutated. The serve path is fast enough for frame-accurate scrubbing: the whole-episode numeric bundle (∼314 KB for 120 frames) is produced in 2.3 ms, and every per-frame payload (rig state, a JPEG camera frame, a lidar frame) serves in under 2 ms.

Keywords: imitation learning, dataset tooling, robot visualization, HDF5, three.js, non-linear editing, EDL.

1 Overview

A teleoperation dataset is only useful if it can be inspected and cleaned. The collector (`../data_collection`) writes one HDF5 file per episode with axis 0 = frame; Data Studio is the read/edit companion. It is a single FastAPI process that serves both a JSON/binary API and a static vanilla-JS / three.js frontend (no build step), reusing the project's canonical robot rig (`mabel_rig.glb` + `mabel_joints.json`) and the live teleop viewer's rendering conventions so the recorded robot animates with exactly the same code path as the live one.

Design constraints. (1) *Single source of truth*: actuator names, camera list, and lidar layout are read from each episode's own `schema_json`, never hard-coded, so the tool cannot drift after a model re-sync. (2) *Non-destructive*: all editing produces new files. (3) *No heavyweight image loads*: dense numeric topics are small and shipped whole; camera and depth frames stream per frame so a long episode never loads gigabytes.

2 Episode data model

Each episode is HDF5 with the keys in Table 1. The studio reads the *dense* numeric keys into one binary bundle and streams the heavy image keys per frame.

Table 1: Episode HDF5 schema consumed by the studio (axis 0 = frame, T frames, 15 Hz). $n_u = 59$ actuators.

key	shape	meaning
`obs/cam/<c>/rgb`	$[T]$ vlen u8	JPEG blob per frame (7 cameras)
`obs/cam/<c>/depth`	$[T, 270, 480]$ u16	metric depth, mm (subset)
`obs/lidar`	$[T, 360]$ f32	planar ranges, m (1°)
`obs/joint_pos,vel,torque`	$[T, 59]$ f32	per-actuator measured state
`action/ctrl`	$[T, 59]$ f32	per-actuator *target* (label)
`obs/qpos,qvel`	$[T, 81], [T, 78]$	full generalized state
`obs/pose/{chassis,head,*_palm}`	$[T, 7]$	world pos + quat (wxyz)
`obs/base_twist`	$[T, 3]$	measured (v_x, v_y, ω_z)
`teleop/*`	various	operator intent (targets, grasp, mode)
`time/sim,wall`	$[T]$ f64	timestamps

2.1 Actuator-to-rig mapping

The recorded state has $n_u = 59$ *actuators* (display names such as `Left Arm 1`, `FL Steer`, `Lift`); the GLB rig exposes 60 *named* joints (snake_case nodes). The map is name normalization

$$\texttt{normalize}(s) = \texttt{lower}(s) \text{ with spaces} \to _,$$

with a single coupling: the one `Lift` actuator drives the two prismatic stages, $q_{\texttt{lift_lower}} = q_{\texttt{lift_upper}} = \frac{1}{2}\, q_{\texttt{Lift}}$, so $59 \to 60$. This is the identical mapping the live teleop bridge uses, so the existing `robot-viewer.js` animates recorded frames unchanged. The base pose comes from `obs/pose/chassis`: with MuJoCo quaternion order (w, x, y, z), the planar yaw is

$$\psi = \operatorname{atan2}\big(2(wz + xy),\, 1 - 2(y^2 + z^2)\big). \tag{1}$$

2.2 Frames and co-registration

The exporter bakes the MuJoCo Z-up → three.js Y-up rotation $Q_{\mathrm{ZUP}} = R_x(-\frac{\pi}{2})$ into the rig's `base_link`. Sensor geometry (lidar, TF triads, EE ghosts) is kept in the native MuJoCo frame inside a `navWorld` group carrying Q_{ZUP}, and the robot composes its base pose on top of the baked transform, so the rig and the map co-register without per-axis fixups. A planar lidar return (ρ_i) at bearing $\theta_i = i \cdot \Delta$ becomes, in the world frame with chassis (x, y, ψ),

$$\begin{pmatrix} X \\ Y \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix} \begin{pmatrix} \rho_i \cos\theta_i \\ \rho_i \sin\theta_i \end{pmatrix}, \tag{2}$$

accumulated over the clip and voxel-downsampled (5 cm) into the map the robot is "mapping" — the requested environment mesh, reconstructed from the episode's own sensing rather than an external scan.

2.3 Camera reconstruction: dense cloud and surface mesh

Following Rerun's *Pinhole + DepthImage* model, a chosen camera (the head ZED or the base forward cam) is back-projected and *fused across keyframes* into one static world-frame model — the map a visual-SLAM front-end (cuVSLAM / nvblox) would build. For a pixel (u, v) with metric

depth d and square-pixel focal length $f = \frac{H/2}{\tan(\text{fovy}/2)}$, the camera-frame point (MuJoCo's $-Z$-forward, $+Y$-up convention) is

$$p_{\text{cam}} = \left(\tfrac{u-c_x}{f}\, d,\ \tfrac{c_y-v}{f}\, d,\ -d\right), \qquad p_{\text{world}} = t_{\text{cam}} + R_{\text{cam}}\, p_{\text{cam}}, \tag{3}$$

with $(t_{\text{cam}}, R_{\text{cam}})$ the per-frame extrinsics (FK on `obs/qpos`, or composed from tracked body poses; same code path as the single-frame clouds, so both register to the rig). Two representations are served over the CLOUD (0x05) and a new MESH (0x06) binary:

- **Dense point cloud** (Rerun *Points3D*): valid points over K keyframes concatenated and voxel-downsampled (2 cm), capped at 220 000.
- **Surface mesh** (Rerun *Mesh3D*): organized-depth triangulation — each 2×2 pixel cell whose four corners are valid *and* depth-consistent (spread $< \delta + 0.03\, d$, so triangles do not stretch across depth discontinuities) emits two triangles; vertices unreferenced by any face are pruned, per-vertex normals are computed, and the mesh is rendered lit (`MeshStandardMaterial`, double-sided) with per-vertex RGB.

Both are one-shot per camera — static as the playhead scrubs. The topic tree is organised by *message type* (Foxglove/Rerun style): a *PointCloud* group lists every depth camera as an on/off cloud source, and a *cuSLAM Mesh* group lists the same cameras as independent mesh ("SLAM cam") sources — each renders its own fused mesh, in the shared 3D viewer. Pure NumPy, no Open3D; 50–160 ms to build on the sample episode ($f = 8$ keyframes).

Two point-cloud pipelines + density. The per-frame depth clouds run in one of two modes, chosen by hostname (overridable with `?hires` / `?remote=1`). *Remote* (the deployed site) fetches one downsampled cloud per frame lazily (throttled), keeping bandwidth sane. *Local* (a localhost server) *prefetches every frame's cloud* on toggle (8-way concurrent, byte-budgeted) so scrubbing/playback is a pure cache hit — the fix for a visible per-frame "blink" when clouds could not be fetched fast enough — at full resolution. Either way the CLOUD binary is bound to the GPU as an *interleaved* buffer (f32 xyz + u8 rgba, 16 B/point) with no per-point JavaScript parse, so even a 56 000-point cloud builds instantly. A *density* toggle *on the 3D viewer* (ultra / high / med / low, a segmented control — a visualisation choice, so it lives with the view, not the topic tree) trades points for speed by setting the pixel stride ($1 \ldots 8$), point budget, and splat size — 56 000 points at ultra vs. ∼800 at low on the sample head cam.

Topics vs. viewer. The right-hand *Topics* panel is the data catalogue — what each message type publishes and which sources to enable (cameras for `Image` / `PointCloud`, SLAM cams for the mesh, robots for `urdf` where `mabel` is one toggle among possibly many). The 3D viewer carries the *visualisation* decisions as grouped chips (Scene / Maps / Camera / Motion), plus *Point Cloud* and *cuSLAM Mesh* dropdown buttons (like "+ Window") to pick which cameras publish each layer directly from the view; the camera-frustum size/distortion and the cloud density (a segmented toggle) sit together in the top-right control cluster.

3 Method: streaming, plots, and editing

Dense bundle. For scrubbing and plots the studio ships all small numeric topics in one little-endian blob: a length-prefixed JSON header (padded to a 4-byte boundary so the browser's `Float32Array` view is aligned) followed by contiguous `f32` blocks. The frontend caches it, so seeking is a pure array index — no network round-trip per frame.

Dense bar-graph. All 59 actuators are drawn as one canvas "equalizer": per-joint value normalized by its episode max-$|\cdot|$, measured as a filled bar and `action/ctrl` as an overlaid target cap, grouped base / lift / neck / arms / hands. Clicking a bar opens its value-over-time strip (measured area + target line + playhead).

EDL and materialization. A timeline is EDL $= \langle (d_k, e_k, a_k, b_k) \rangle$, half-open source segments. Every editing operation is a pure list transform: *trim* sets a/b; *blade* at output frame f splits the active segment; *ripple-delete* drops a segment (the magnetic track closes the gap); *merge* coalesces adjacent contiguous same-source segments; *append* pushes segments from another episode. *Save* materializes the EDL into a new `episode_XXXX.h5` by bulk-slicing every key (vlen JPEG copied losslessly), with the union-of-keys filled where a source lacks a key, and `time/*` rebased to be strictly monotonic across joins:

$$t_0 = 0, \qquad t_n = t_{n-1} + \Delta_n, \quad \Delta_n = \begin{cases} t_n^{\text{src}} - t_{n-1}^{\text{src}} & \text{within a segment} \\ 1/\text{fps} & \text{at a segment join.} \end{cases} \tag{4}$$

Source files are opened read-only; the writer reuses the recorder's dtype and gzip conventions, and publishes atomically via a `.tmp`→rename.

Editing power-features (FCP/Rerun-inspired). On top of the segment EDL the timeline adds non-destructive operations aimed at *cleaning* and *aligning* demonstrations, all applied only at materialization:

- **Reject/Keep ranges** ($\langle s, e, \text{kind} \rangle$ over output frames). A *reject* span is ripple-*removed* from every modality before concatenation (a global keep-mask $K \in \{0,1\}^T$, output length $T' = \sum K$), excising fumbled takes in one gesture; annotations are remapped through the prefix sum of K.
- **Per-modality time offset** o_m (J/L-cut): modality m reads source frame $f - o_m$ (edge-held), correcting camera-vs-proprioception latency.
- **Retime** r per clip: nearest-neighbour resample to $\lceil n/r \rceil$ frames at uniform fps — normalise rate, slow-mo, or skip idle.
- **Auto event markers** (grasp / mode / motion transitions detected on the dense bundle) and a **QC** pass (NaNs, clip-join discontinuities $\|q_{\text{first}(B)} - q_{\text{last}(A)}\|_\infty > \tau$) surface cut points and defects; a per-clip signal waveform makes events scannable.

These extend the segment tuple to $(d_k, e_k, a_k, b_k, \text{modkeep}, \text{modoffset}, r_k)$; `edl.normalize` must carry every per-segment field through or it is silently dropped before the writer.

4 Experiments

We benchmark the serve path against three real sample episodes ($T = 120$, seven cameras) generated by the headless mock-AVP collector (`sampledata.py`), regenerated from scratch by `python -m data_studio.server.bench_studio`; Fig. 1.

A representative non-destructive edit — trim each of two episodes to their first half and concatenate ($120 \to 180$ output frames in another run) — materialized in $\sim$1.4 s, dominated by copying $\sim 180 \times 7$ JPEG blobs; the result reads back with strictly monotonic time and all image streams intact. The dense numeric payload scales linearly in T and stays in the few-hundred-KB range (Fig. 1, right), which is why the whole episode can be fetched once and scrubbed offline.

Table 2: Serve benchmark, mean over the sample dataset (single process, in-process calls). Per-payload times are well under one display frame ($1/15 \approx 67$ ms), so scrubbing is instant.

	h5 (MB)	bundle (KB)	bundle	frame state	JPEG frame	lidar	scene
120 frames	24.2	313.7	2.3 ms	1.7 ms	0.2 ms	0.8 ms	3.5 ms

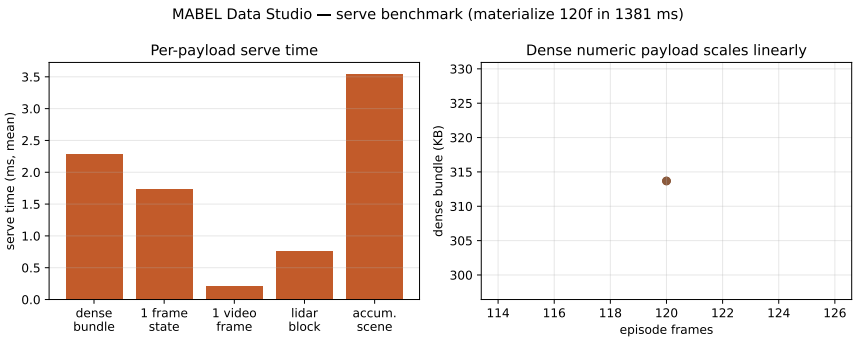


Figure 1: Left: mean serve time per payload type. Right: dense numeric bundle size vs. episode length. Generated by `bench_studio.py`.

5 Implementation & usage

`server/` is a FastAPI app: `catalog` (discovery), `episode` (meta / bundle / frame / lidar), `video` (JPEG / MJPEG / cached MP4 / depth), `scene` (lidar clouds + accumulated map), `edl` / `materialize` / `projects` (editing), `camcloud` (camera intrinsics, FK extrinsics, depth→cloud, and the fused reconstruction cloud / mesh), `sampledata` (seed data), over an LRU `h5pool`. `web/` is vanilla ES modules: `viewport3d` (vendored `Stage`/`Rig`/Q_{ZUP}), `videowall`, `bargraph`, `timeline`, `sequence` (the EDL model), `topics`, `browser`, driven by one global-frame display loop. Run:

```
cd learning/imitation_learning
python -m data_studio.server          # http://127.0.0.1:8092
python -m data_studio.server.sampledata --episodes 3   # seed data
python -m data_studio.server.camcloud --dataset NAME   # build camera caches
```

For a *model-less deployment* (e.g. the public host has neither MuJoCo nor the ∼300 MB compiled scene), `camcloud` reads a tiny per-episode `<episode>.camcache.npz` sidecar (camera intrinsics + per-frame world poses) and skips MuJoCo entirely; the depth→cloud unprojection is pure NumPy. A MuJoCo host computes these live *and* writes the sidecar, so dev behaviour is unchanged — ship the sidecars next to the episodes.

6 Limitations

The accumulated map is only as rich as the episode's motion; a near-stationary mock run yields a sparse cloud (real driving data fills it in). Per-camera extrinsics and intrinsics (f_y from `cam_fovy`) are recovered by forward kinematics on `obs/qpos` rather than stored in the schema, so a dataset collected on hardware without the MuJoCo model would need its camera cache built elsewhere (one `camcloud --dataset` pass on any MuJoCo host). MP4 transcoding shells out to `ffmpeg` (the

primary scrub path uses the stored JPEG blobs directly and needs no transcode). Materialize cost is dominated by JPEG-blob copying, linear in output frames.

References

[1] Rerun.io, *Rerun: an SDK for visualizing multimodal data*, 2024.

[2] Quigley et al., *ROS: an open-source Robot Operating System*, ICRA Workshop, 2009. (`rviz`.)

[3] The HDF Group, *Hierarchical Data Format, version 5*.

[4] Zhao et al., *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT), RSS 2023.